

Putting Evals Into Your Agent Runtime" / X

 x.com/JoshARosen/status/2097324183428444499

2026年9月8日

LLM-as-Judge Architectures: Putting Evals Into Your Agent Runtime

LLM 作为评判架构：将评估放入您的代理运行时

LLM-as-judge is already a common part of building AI applications. Run your application against a dataset, have another model grade the results, and use the scores to figure out whether the latest version got better or worse.

将 LLM 作为评判标准已成为构建 AI 应用的常见做法。只需将应用运行在数据集上，让另一个模型对结果进行评分，然后利用这些分数来判断最新版本是改进了还是退步了即可。

Now, LLM judges are moving into the runtime itself, where they evaluate work as it happens. Increasingly, we are seeing products put LLM judges directly inside the agent loop rather than using them as an offline tool. The judge is no longer just evaluating the application. It's participating in its control flow.

现在，LLM 评判器正进入运行时环境，实时评估应用程序的运行情况。我们看到越来越多的产品将 LLM 评判器直接置于代理循环中，而不是将其作为离线工具使用。评判器不再仅仅是评估应用程序，而是参与到应用程序的控制流程中。

If your application does not have some form of LLM judge yet, there is a good chance it eventually will. Traditional deterministic software gives us plenty of ways to check whether something worked. We can use schema validation, type checks, assertions, and exact comparisons.

如果你的应用目前还没有某种形式的 LLM（逻辑推理）判断机制，那么它最终很可能会加入。传统的确定性软件为我们提供了许多方法来检查某个功能是否正常工作。我们可以使用模式验证、类型检查、断言和精确比较。

AI applications, on the other hand, produce work that often cannot be checked that way. Is the work complete enough to move on? Was the answer actually useful? Did the research support the conclusion? Should I let this generated artifact into my company brain? If a human would normally have to look at the result and make a judgment call, an LLM judge gives the application a way to make some of those judgments itself.

另一方面，人工智能应用生成的成果往往无法用这种方式进行检验。这项工作是否足够完善，可以继续下一步？答案是否真的有用？研究是否支持结论？我是否应该将生成的成果纳入公司决策体系？通常情况下，需要由人类来查看结果并做出判断，而 LLM 评审员则赋予了应用程序自行做出部分判断的能力。

The model evaluation world has spent years figuring out how to get models to reliably judge other models. But it turns out that many of these techniques can also be useful one layer higher, as part of the architecture of the applications we are building with those

models.

模型评估领域多年来一直在探索如何让模型可靠地判断其他模型。但事实证明，其中许多技术在更高层次上也同样适用，即作为我们使用这些模型构建的应用程序架构的一部分。

There are a ton of ways to build that layer. Here are some common patterns for LLM-as-judge that you should consider adopting as part of your product's application architecture. 构建这一层的方法有很多。以下是一些常见的 LLM 作为评判者的模式，您可以考虑将其纳入产品应用架构中。

Start With Another Model 从另一个模型开始

This is the canonical LLM-as-judge approach. One model does the work and another model judges it. The judge gets some combination of the original request, the generated result, a rubric, reference material, and perhaps an expected answer. It returns a score, label, or explanation. Systems such as

[LangSmith](#)

,

[Phoenix](#)

, and

[DeepEval](#)

make this a normal part of application evaluation.

这是典型的 LLM 作为评判者的方法。一个模型执行任务，另一个模型进行评判。评判者会收到原始请求、生成结果、评分标准、参考资料以及可能存在的预期答案等信息。它会返回分数、标签或解释。诸如此类的系统

[朗史密斯](#)

,

[凤凰](#)

, 和

[深度评估](#)

将其作为申请评估的常规环节。

One upfront choice you have to make with this approach: which model should be the judge? If cost were no concern, the obvious answer is to use your strongest frontier model. But the judge does not necessarily need to be smarter in the general sense. Judging a narrow property can be much easier than doing the original work. A model that could not produce a great research report may still be perfectly capable of deciding whether every claim is supported by the supplied evidence.

采用这种方法，你必须预先做出一个选择：应该使用哪个模型来评判？如果成本不是问题，显而易见的答案是使用你最强大的前沿模型。但评判模型未必需要在总体上更智能。评判一个具体的属性可能比进行原始研究容易得多。一个无法生成出色研究报告的模型，仍然完全能够判断每个论断是否都得到了所提供证据的支持。

This has led to specialized judge models such as

[Prometheus](#)

, as well as smaller evaluators trained or distilled for particular kinds of judgment.

[Galileo](#)

has also built specialized evaluation models intended to make production-scale evaluation cheaper than repeatedly calling a large frontier model.

这催生了诸如此类的专门裁判模式。

[普罗米修斯](#)

以及接受过特定类型判断培训或精简的小型评估人员。

[伽利略](#)

还构建了专门的评估模型，旨在降低生产规模评估的成本，使其比反复调用大型前沿模型更便宜。

For high-volume applications, the economics could matter quite a bit. Because cost is more noticeable when you're running these evals constantly at application runtime, a specialized model that is designed to be cheap for this use case is a good option.

对于高容量应用而言，经济效益至关重要。因为在应用运行时持续进行这些评估时，成本会更加明显，所以针对这种用例设计的低成本专用模型是一个不错的选择。

Break the Judgment Apart

打破评判

A large amount of LLM judging boils down to one main question: was this output good? An answer can be correct but incomplete. It can be well written but unsupported by the source material. Or an agent can arrive at the right result after taking actions it should never have taken.

大量的 LLM（逻辑推理模型）评判最终都归结为一个核心问题：这个输出结果好不好？答案可能正确但不完整，也可能写得很好但缺乏原始资料的支持，或者智能体采取了不该采取的行动后却得出了正确的结果。

One upgrade to your judging architecture is to break the judgment into a set of smaller decisions. One judge checks whether the answer addresses the request. Another checks whether its claims are supported by the supplied evidence. Another looks at whether the agent completed the required work.

对评判体系的一种改进是将评判过程分解成一系列更小的决策。一位评判员检查答案是否回应了请求。另一位评判员检查答案的主张是否有证据支持。还有一位评判员检查代理人是否完成了所需的工作。

[G-Eval](#)

took an early step in this direction by having the model generate evaluation steps from the criteria before producing its score.

[DeepEval](#)

pushes the idea further with DAG-based evaluation, where individual LLM judgments can sit inside a larger deterministic decision graph.

[G-Eval](#)

该模型在生成分数之前，先根据标准生成评估步骤，从而朝着这个方向迈出了第一步。

[深度评估](#)

通过基于 DAG 的评估进一步推进这一想法，其中单个 LLM 判断可以位于更大的确定性决策图中。

In other words, instead of asking one model to make a giant fuzzy decision, break that decision into smaller judgments with well crafted smaller prompts and put structure around how they are combined.

换句话说，与其要求一个模型做出一个巨大的模糊决定，不如将该决定分解成更小的判断，通过精心设计的较小提示，并围绕如何组合这些判断构建结构。

Compare Instead of Score 比较而非评分

Models are not always particularly good at telling you that something deserves a 7 rather than an 8. They can be much better at deciding which of two things is better. Pairwise judging is an approach that takes advantage of this. If you give the judge two outputs produced from the same input, you can ask which one better satisfies the criteria.

模型并不总是擅长告诉你某个东西应该得7分而不是8分。但它们在判断两个事物哪个更好方面却表现出色得多。成对评判法正是利用了这一点。如果你给评判者两个由同一输入产生的输出，你可以询问哪个输出更符合评判标准。

Systems such as

[LangSmith](#)

and

[DeepEval](#)

support this directly, including techniques such as randomizing which answer appears first to reduce position bias.

例如这样的系统

[朗史密斯](#)

和

[深度评估](#)

直接支持这一观点，包括采用随机化答案显示顺序等技术来减少位置偏差。

This is commonly useful for application regression testing. However, the same pattern could move into runtime architectures. An agent could generate several plans and use a judge to choose between them. Alternatively, two agents could independently perform a piece of analysis and another model could compare the results. In short, a proposed action could be compared against an alternative before the system commits to it and proceeds.

这在应用程序回归测试中通常很有用。然而，同样的模式也可以应用于运行时架构。一个代

理可以生成多个方案，并使用评判器从中选择。或者，两个代理可以独立执行分析，然后另一个模型可以比较结果。简而言之，在系统最终确定并执行某个方案之前，可以将该方案与备选方案进行比较。

Judge the Work Instead of the Answer 评判作品本身，而不是答案。

For a chatbot, judging the final response is fine. For an agent doing twenty minutes of work, it tells you much less. The final result might look completely reasonable even though the agent retrieved the wrong documents or ignored an important source. It may have called the wrong tool or wandered through a series of unnecessary steps before getting lucky at the end.

对于聊天机器人来说，判断最终回复是可以的。但对于一个工作了二十分钟的客服人员来说，这种判断就意义不大。即使客服人员检索了错误的文档或忽略了重要的信息源，最终结果看起来也可能完全合理。它可能调用了错误的工具，或者在最后侥幸成功之前，走了不少弯路。

Application evaluation systems are already moving further into the trace.

[Phoenix](#)

has evaluators for retrieval relevance, tool selection, tool invocation, tool responses, and overall agent performance.

[LangSmith](#)

can apply evaluators to individual runs as well as larger traces and threads.

应用评估系统已经开始向追踪层面发展。

[凤凰](#)

具有检索相关性、工具选择、工具调用、工具响应和整体代理性能的评估器。

[朗史密斯](#)

可以将评估器应用于单个运行以及更大的跟踪和线程。

Generalizing this to agents in general and especially long-running agents, the takeaway is to judge the pieces of work where the important decisions are actually being made.

将此推广到一般的代理人，特别是长期代理人，其结论是要评判那些真正做出重要决策的工作。

For example, a research agent might have its source selection judged before synthesis. Or a coding agent might have its proposed approach judged before implementation. An operational agent might have the evidence supporting an action judged before that action is taken. Integrating this into your application means scattering judges throughout the runtime to form key checkpoints.

例如，研究代理的资料来源选择可能在综合分析之前接受评估；编码代理的方案提出可能在实施之前接受评估；操作代理的行动证据可能在执行该行动之前接受评估。将此功能集成到应用程序中意味着在整个运行时环境中分散设置评估点，以形成关键的检查点。

Use More Than One Judge

使用多名法官

One uncomfortable fact about LLM-as-judge is that the judge is still an LLM. It can make mistakes for all the same reasons the model doing the work can. One way to handle this is to stop treating a single judge as authoritative.

关于“LLM 作为评判者”这一概念，一个令人不安的事实是，评判者本身仍然是一个 LLM 模型。它可能会犯错，原因与执行任务的模型相同。解决这个问题方法之一是停止将单个评判者视为权威。

Research on model evaluation has explored panels of judges, evaluator personas, voting, aggregation, and debate between evaluators.

[MAJ-EVAL](#)

, for example, creates multiple evaluator agents representing different dimensions of the evaluation and lets them deliberate over the result.

模型评估研究探索了评委小组、评估者角色、投票、汇总以及评估者之间的辩论。

[重要评估](#)

例如，创建多个评估代理来代表评估的不同维度，并让他们对结果进行讨论。

There is not a lot of evidence suggesting applications have adopted this approach in production yet. But the mechanism is interesting above the model layer for a different reason. Even if you don't care whether three judges vote 2–1 that an answer is good, you may care enormously that they disagree.

目前还没有太多证据表明应用程序已经在生产环境中采用了这种方法。但这种机制在模型层之上却因另一个原因而引人注目。即使你不在乎三位评委以 2 比 1 的投票结果是否认为某个答案正确，你也可能非常在意他们意见不一致的情况。

Agreement between independent judgments can increase confidence whereas disagreement could be a reason to retry with a stronger model, collect more evidence, or send the work to a human. You could imagine building a retry or escalation mechanism in your app based on this principle.

独立判断结果一致可以增强信心，而分歧则可能意味着需要使用更强大的模型进行重试、收集更多证据，或者将工作交给人工处理。您可以考虑基于此原则，在您的应用程序中构建重试或升级机制。

Judge the Judge

法官

Once a judge can affect what happens in an application, its reliability matters a lot. And things can go wrong. For example, LLM judges have been known to have biases such as changing their decisions based on which answer is presented first. Or they can favor certain writing styles. They might struggle when the outputs being evaluated are close in

quality. Sometimes a model might even have preferences for outputs that resemble its own.

一旦评委能够影响申请结果，其可靠性就至关重要。然而，也可能出现问题。例如，LLM（法学硕士）评委就存在偏见，例如根据答案的先后顺序改变判决。他们也可能偏爱某些写作风格。当评估的输出结果质量相近时，评委可能难以做出准确判断。有时，模型甚至可能偏好与自身相似的输出结果。

The model-evaluation world has built benchmarks specifically to measure these problems.

[LLMBar](#)

tests whether judges can distinguish instruction-following responses under difficult conditions.

[JudgeBench](#)

contains difficult response pairs with objective preferences.

[RewardBench](#)

evaluates reward models across challenging preference tasks and can also be used to evaluate generative LLM judges.

模型评估领域已经建立了专门衡量这些问题的基准。

[LLMBar](#)

测试评委在困难条件下能否区分符合指令的回答。

[法官席](#)

包含具有客观偏好的困难回答对。

[奖励台](#)

评估具有挑战性的偏好任务中的奖励模型，也可用于评估生成式 LLM 评判员。

Anthropic's

[Bloom](#)

provides another useful pattern. Anthropic evaluated a collection of candidate judge models against human-labeled transcripts before choosing its judge. The resulting evaluation pipeline also includes a meta-judge that looks across the broader evaluation results.

人种学派

[盛开](#)

另一种有用的模式是，Anthropic 在最终选定评判员之前，先用人工标注的文本对一系列候选评判员模型进行了评估。由此产生的评估流程还包括一个元评判员，用于分析更广泛的评估结果。

Application builders can use the simpler version of this idea today: periodically compare the judge against humans. If a judgment controls something important, collect examples of those decisions and have people independently evaluate them. Measure where the judge disagrees, and change the rubric, model, context, or decision boundary when the disagreement is unacceptable.

应用程序开发者现在就可以使用这个理念的简化版本：定期将系统判别器与人类判别器进行比较。如果某个判别器判断出的结果至关重要，那就收集这些判别器的案例，并让其他人独

立评估它们。找出系统判别器与人类判别器判断不一致的地方，并在分歧无法接受时，修改评分标准、模型、上下文或判别边界。

Put Determinism Around the Judge 将决定论强加于法官身上

There is a temptation to turn every application decision into another LLM judgment and to throw more agents at more agents. That dismisses one of the biggest advantages we have at the application layer that, unlike the model, we control the surrounding system.

人们很容易将每个应用决策都变成另一个 LLM（逻辑逻辑模型）判断，并让更多的智能体去对抗更多的智能体。这样做忽略了我们在应用层拥有的最大优势之一：与模型不同，我们可以控制周围的系统。

If something can be checked deterministically, check it deterministically with tests or schema checks or even a database check. You could also consider writing a policy engine for this type of validation.

如果某件事可以确定性地进行检查，那就用测试、模式检查甚至数据库检查来确定性地检查它。你也可以考虑为这类验证编写策略引擎。

[OpenAI's grader architecture](#)

reflects this separation. It supports model-based graders alongside deterministic graders such as string checks and Python code, and multiple graders can be combined into a larger evaluation.

[OpenAI 的评分器架构](#)

这种分离体现在两个方面：一是支持基于模型的评分器，二是支持基于确定性的评分器（例如字符串检查和 Python 代码），并且可以将多个评分器组合起来进行更全面的评估。

The same pattern makes sense inside applications. An LLM might judge whether the evidence is sufficient, whether a recommendation is well supported, or whether an agent appears to have completed the task. Deterministic code can decide what combination of those judgments and other facts is required before the workflow moves forward.

同样的模式也适用于应用程序内部。逻辑逻辑模型（LLM）可以判断证据是否充分、建议是否得到充分支持，或者代理是否已完成任务。确定性代码可以决定在工作流程继续之前，需要结合哪些判断和其他事实。

This sort of combined deterministic / non-deterministic logic is one of the biggest opportunities available at the application layer. The better we get at drawing that boundary, the more value we are adding above and beyond the models themselves.

这种确定性/非确定性逻辑的结合是应用层最大的机遇之一。我们越能精准地划定二者之间的界限，就能在模型本身之外创造越多的价值。

Implication: Put the Judge in the Loop

含义：让法官参与进来

Many of the above patterns require that we move LLM judges inside the agent loops and put them in the critical path for our applications. That's in contrast to the many LLM judges today that sit outside the application execution path.

上述许多模式都要求我们将 LLM 评判器移到代理循环内部，并将其置于应用程序的关键路径上。这与目前许多 LLM 评判器位于应用程序执行路径之外的情况截然不同。

It means that judgment informs control flow. It can cause the application to continue, retry, route to another model, gather more information, or escalate to a person. This is a much bigger role for LLM-as-judge. The judge is no longer just telling you whether the application worked yesterday. It is helping determine what the application does next.

这意味着判断会影响控制流程。它可以决定应用程序继续运行、重试、路由到其他模型、收集更多信息，或者升级给相关人员。这使得 LLM 作为判断者的角色更加重要。判断者不再仅仅是告诉你应用程序昨天是否运行正常，它还帮助决定应用程序下一步该做什么。

It also means mistakes from the judge become application failures. A judge that is slightly noisy in an offline eval may be annoying. The same judge sitting in front of every important action can create loops, block good work, approve bad work, and add latency to every execution.

这也意味着评判员的错误会导致应用程序失败。离线评估中，评判员哪怕只是发出一些噪音都可能令人烦恼。如果同一个评判员出现在每个重要操作的前面，则可能导致循环、阻碍好的工作、批准不好的工作，并增加每次执行的延迟。

We will need to invest in these architectures and mature these patterns before we feel comfortable putting LLM judges on the critical path in production. But there are huge opportunities for innovation here.

我们需要对这些架构进行投入，并完善这些模式，才能放心地将 LLM 评测器部署到生产的关键路径上。但这里面蕴藏着巨大的创新机遇。